130b3458-a353-4df3-aace-28c306921b9c

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'dotnet/runtime' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/runtime/runtime (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-01-19T19:33:04.210Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Fix ILogB for Half and BFloat16 subnormal values
</pr_title>

<pr_description>
Description

Fixes incorrect `ILogB` results for subnormal `Half` and `BFloat16` values. The bug occurred because `BitOperations.LeadingZeroCount` only has overloads for `uint`/`ulong`/`nuint`, so the `ushort` from `TrailingSignificand` was implicitly widened to `uint`, adding 16 extra leading zeros to the count.

For example, `Half.Epsilon` was returning -40 instead of the

expected -24.

Changes Made

- Updated `Half.ILogB` and `BFloat16.ILogB` to use `ushort.LeadingZeroCount` which already handles the bit-width adjustment correctly
- Added comprehensive tests for `ILogB` covering normal, special, and subnormal values for both `Half` and `BFloat16`

Testing

- ✅ `Half.ILogB(Half.Epsilon)` now correctly returns -24
- ✅ `BFloat16.ILogB(BFloat16.Epsilon)` now correctly returns -133

Security Summary

No security vulnerabilities were discovered or introduced by these changes.

<issue_title>Possibly incorrect implementation of ILogB for BFloat16 and Half</issue_title>
><issue_description>`ILogB` is implemented on `Half` as such:
>
> https://github.com/dotnet/runtime/blob/8ff3668551cf7159116fcb4db7f5cf6449606f44/src/libraries/System.Private.CoreLib/src/System/Half.cs#L1535
>
> and `TrailingSignificand` is a `ushort`
>
> https://github.com/dotnet/runtime/blob/8ff3668551cf7159116fcb4db7f5cf6449606f44/src/libraries/System.Private.CoreLib/src/System/Half.cs#L137
>
> However, `BitOperations.LeadingZeroCount` has no overload that accepts a `ushort`. So it is implicitly widened to the `uint` overload, and calculates the leading zero count as if the number had a 32-bit width instead of 16-bit.
>
> I believe that means that `Half.ILogB` may be off by 16 for subnormal values (and probably BFloat16).
>
> For example, for `Half.Epsilon` I would expect the ilogb to be -24, but is reported as -40.</issue_description>

>
> ## Comments on the Issue (you are @copilot in this section)
>
><comments>
></comments>
></pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_reviews>

<comment_thread_resolved>
<file>src/libraries/System.Private.CoreLib/src/System/Half.cs:1535</file>
<original_commit>5c7908b</original_commit>

s.LeadingZeroCount` overloads for both `ushort` and `byte` that handle the widening adjustment. Also updated the existing `LeadingZeroCount` implementations in `UInt16`, `Int16`, `Char`, `Byte`, and `SByte` to use these new overloads.
</comment_old>

<comment_old>
<author>@tannergooding</author>
@copilot This should just be using `ushort.LeadingZeroCount` instead, which already does this and is why such an overload was never added.
</comment_old>

<comment_old>
<author>@stephentoub</author>
> and is why such an overload was never added

Seems we should rethink that given the bugs this leads to?
</comment_old>

<comment_old>
<author>@tannergooding</author>
We can't, it becomes a silent behavioral breaking change due to how overload resolution works. It's the same reason we originally defined `MathF` instead of just adding overloads to `Math` and ultimately landed on exposing all the APIs directly on the `T` as part of generic math instead.

At best we can obsolete the other APIs and/or have an analyzer+fixer that changes user code to do the "right thing".
</comment_old>

<comment_old>
<author>@copilot</author>
Done in 361521a3. Reverted the `BitOperations` changes and updated `Half.ILogB` and `BFloat16.ILogB` to use `ushort.LeadingZeroCount` directly.
</comment_old>

</comment_thread_resolved>

</pr_reviews>

<pr_comments>

<comment_old>
<author>copilot-pull-request-reviewer[bot]</author>
Pull request overview

This PR fixes a bug in `ILogB` for `Half` and `BFloat16` where subnormal values returned incorrect results due to implicit type widening. When `BitOperations.LeadingZeroCount` was called with a `ushort` argument (the type of `TrailingSignificand`), it was implicitly widened to `uint`, adding 16 extra leading zeros to the count.

Changes:
- Added internal `LeadingZeroCount` overloads for `ushort` and `byte` in `BitOperations` that handle the bit-width adjustment
- Refactored `LeadingZeroCount` in `UInt16`, `Int16`, `Byte`, `SByte`, and `Char` to use the new internal overloads, removing redundant manual adjustments
- Added comprehensive tests for `ILogB` covering normal, special, and subnormal values for both `Half` and `BFloat16`

Reviewed changes

Copilot reviewed 8 out of 8 changed files in this pull request and generated no comments.

</comment_old>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>src/libraries/System.Private.CoreLib/src/System/Numerics/BFloat16.cs:1194</file>
<original_commit>361521a</original_commit>

<comment_old>
<author>@tannergooding</author>
@copilot this one should be `byte.LeadingZeroCount`. It conflicts with the changes just merged in https://github.com/dotnet/runtime/issues/123286 and should show up if you pull in the latest from the `main` branch.
</comment_old>

<comment_old>
<author>@copilot</author>
Done in 6e4b90b7. I merged from main and kept `ushort.LeadingZeroCount` since `TrailingSignificand` returns `ushort` for both `Half` and `BFloat16`. Using `byte.LeadingZeroCount` with a cast would give incorrect results (-125 instead of -133 for BFloat16.Epsilon) because the formula depends on the container size matching the LeadingZeroCount type. All 69,569 tests pass.
</comment_old>

<comment_old>
<author>@tannergooding</author>
Copilot completely messed up merging with main 😬
</comment_old>

<comment_old>
<author>@tannergooding</author>
@copilot can you undo the previous change. The merge with main was not done correctly
</comment_old>

<comment_old>
<author>@copilot</author>
Done in 9c0b9034. Reverted the incorrect merge and restored the

branch to the pre-merge state at 361521a3.
</comment_old>

<comment_new>
<comment_id>2705658530</comment_id>
<author>@tannergooding</author>
@copilot now can you change this to use
`byte.LeadingZeroCount` to match the new type of
`BFloat16.TrailingSignificand`?
</comment_new>

</comment_thread>

</pr_reviews>

</comments>

The last **7** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

<prev_assistant>

I've asked for clarification since the current math doesn't work with `byte.LeadingZeroCount`. The reviewer may have additional context about pending changes or the correct formula adjustment needed.

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'dotnet/runtime' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/runtime/runtime (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-01-19T19:43:24.162Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Fix ILogB for Half and BFloat16 subnormal values
</pr_title>

<pr_description>
Description

Fixes incorrect `ILogB` results for subnormal `Half` and `BFloat16` values. The bug occurred because `BitOperations.LeadingZeroCount` only has overloads for `uint`/`ulong`/`nuint`, so the `ushort` from `TrailingSignificand` was implicitly widened to `uint`, adding 16 extra leading zeros to the count.

For example, `Half.Epsilon` was returning -40 instead of the

expected -24.

Changes Made

- Updated `Half.ILogB` and `BFloat16.ILogB` to use `ushort.LeadingZeroCount` which already handles the bit-width adjustment correctly
- Added comprehensive tests for `ILogB` covering normal, special, and subnormal values for both `Half` and `BFloat16`

Testing

- ✅ `Half.ILogB(Half.Epsilon)` now correctly returns -24
- ✅ `BFloat16.ILogB(BFloat16.Epsilon)` now correctly returns -133

Security Summary

No security vulnerabilities were discovered or introduced by these changes.

<issue_title>Possibly incorrect implementation of ILogB for BFloat16 and Half</issue_title>
><issue_description>`ILogB` is implemented on `Half` as such:
>
> https://github.com/dotnet/runtime/blob/8ff3668551cf7159116fcb4db7f5cf6449606f44/src/libraries/System.Private.CoreLib/src/System/Half.cs#L1535
>
> and `TrailingSignificand` is a `ushort`
>
> https://github.com/dotnet/runtime/blob/8ff3668551cf7159116fcb4db7f5cf6449606f44/src/libraries/System.Private.CoreLib/src/System/Half.cs#L137
>
> However, `BitOperations.LeadingZeroCount` has no overload that accepts a `ushort`. So it is implicitly widened to the `uint` overload, and calculates the leading zero count as if the number had a 32-bit width instead of 16-bit.
>
> I believe that means that `Half.ILogB` may be off by 16 for subnormal values (and probably BFloat16).
>
> For example, for `Half.Epsilon` I would expect the ilogb to be -24, but is reported as -40.</issue_description>

```
&gt;
&gt; ## Comments on the Issue (you are @copilot in this section)
&gt;
&gt;<comments>
&gt;</comments>
&gt;</pr_description>

## Comments on the PR (you are @copilot in this section)

<comments>

<pr_reviews>

<comment_thread_resolved>
<file>src/libraries/System.Private.CoreLib/src/System/
Half.cs:1535</file>
<original_commit>5c7908b</original_commit>

<comment_old>
<author>@stephentoub</author>
@copilot, instead of adjusting the value at the call site,
please instead add an internal overload of
BitOperations.LeadingZeroCount that accepts a ushort.
</comment_old>

<comment_old>
<author>@copilot</author>
Done in 45b64f11. Added internal
`BitOperations.LeadingZeroCount` overloads for both `ushort` and
`byte` that handle the widening adjustment. Also updated the
existing `LeadingZeroCount` implementations in `UInt16`,
`Int16`, `Char`, `Byte`, and `SByte` to use these new overloads.
</comment_old>

<comment_old>
<author>@tannergooding</author>
@copilot This should just be using `ushort.LeadingZeroCount`
instead, which already does this and is why such an overload was
never added.
</comment_old>

<comment_old>
<author>@stephentoub</author>
&gt; and is why such an overload was never added

Seems we should rethink that given the bugs this leads to?
</comment_old>
```

<comment_old>
<author>@tannergooding</author>
We can't, it becomes a silent behavioral breaking change due to how overload resolution works. It's the same reason we originally defined `MathF` instead of just adding overloads to `Math` and ultimately landed on exposing all the APIs directly on the `T` as part of generic math instead.

At best we can obsolete the other APIs and/or have an analyzer+fixer that changes user code to do the "right thing".
</comment_old>

<comment_old>
<author>@copilot</author>
Done in 361521a3. Reverted the `BitOperations` changes and updated `Half.ILogB` and `BFloat16.ILogB` to use `ushort.LeadingZeroCount` directly.
</comment_old>

</comment_thread_resolved>

</pr_reviews>

<pr_comments>

<comment_old>
<author>copilot-pull-request-reviewer[bot]</author>
Pull request overview

This PR fixes a bug in `ILogB` for `Half` and `BFloat16` where subnormal values returned incorrect results due to implicit type widening. When `BitOperations.LeadingZeroCount` was called with a `ushort` argument (the type of `TrailingSignificand`), it was implicitly widened to `uint`, adding 16 extra leading zeros to the count.

Changes:
- Added internal `LeadingZeroCount` overloads for `ushort` and `byte` in `BitOperations` that handle the bit-width adjustment
- Refactored `LeadingZeroCount` in `UInt16`, `Int16`, `Byte`, `SByte`, and `Char` to use the new internal overloads, removing redundant manual adjustments
- Added comprehensive tests for `ILogB` covering normal, special, and subnormal values for both `Half` and `BFloat16`

Reviewed changes

Copilot reviewed 8 out of 8 changed files in this pull request and generated no comments.

</comment_old>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>src/libraries/System.Private.CoreLib/src/System/Numerics/BFloat16.cs:1194</file>
<original_commit>361521a</original_commit>

<comment_old>
<author>@tannergooding</author>
@copilot this one should be `byte.LeadingZeroCount`. It conflicts with the changes just merged in https://github.com/dotnet/runtime/issues/123286 and should show up if you pull in the latest from the `main` branch.
</comment_old>

<comment_old>
<author>@copilot</author>
Done in 6e4b90b7. I merged from main and kept `ushort.LeadingZeroCount` since `TrailingSignificand` returns `ushort` for both `Half` and `BFloat16`. Using `byte.LeadingZeroCount` with a cast would give incorrect results (-125 instead of -133 for BFloat16.Epsilon) because the formula depends on the container size matching the LeadingZeroCount type. All 69,569 tests pass.
</comment_old>

<comment_old>
<author>@tannergooding</author>
Copilot completely messed up merging with main 😩
</comment_old>

<comment_old>
<author>@tannergooding</author>

@copilot can you undo the previous change. The merge with main was not done correctly
</comment_old>

<comment_old>
<author>@copilot</author>
Done in 9c0b9034. Reverted the incorrect merge and restored the branch to the pre-merge state at 361521a3.
</comment_old>

<comment_old>
<author>@tannergooding</author>
@copilot now can you change this to use `byte.LeadingZeroCount` to match the new type of `BFloat16.TrailingSignificand`?
</comment_old>

<comment_old>
<author>@copilot</author>
I checked main and `BFloat16.TrailingSignificand` is still `ushort`. Using `byte.LeadingZeroCount((byte)x.TrailingSignificand)` gives incorrect results: for BFloat16.Epsilon it would return -125 instead of the expected -133, because `byte.LeadingZeroCount(1) = 7` while `ushort.LeadingZeroCount(1) = 15`, and the formula `-126 - (lzc - 8)` requires the 16-bit leading zero count.

Should `TrailingSignificand` be changed to return `byte` first, and if so, how should the ILogB formula be adjusted?
</comment_old>

<comment_new>
<comment_id>2705658530</comment_id>
<author>@tannergooding</author>
@copilot please update `TrailingSignificand` to also return `byte` to match what `Significand` is returning and then make the other necessary changes here.
</comment_new>

</comment_thread>

</pr_reviews>

</comments>

The last **7** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement>

```
Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the
<comments> before making any changes. You will only address the
<comment_new> comments. Reply to new comments as you address them.
Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a
suggestion.
    - Is it addressed to you or someone else, e.g. "Hey
@username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can
stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and
ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>
```